1. **Aim:** Create attendance module for students and teachers.

2. **Task to be done:**
   - To design and implement an attendance tracking system where:
   - Teachers can mark attendance.
   - Students can view their own attendance.
   - The data is securely stored and associated with course/module info.

3. **Code for experiment:**

```
const express = require('express'); const
cors = require('cors');
const mongoose = require('mongoose');
require('dotenv').config();

const app = express();

// Middleware - ORDER MATTERS!
app.use(cors()); app.use(express.json());
app.use(express.urlencoded({ extended: true }));
// Database Connection
mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/attendance_db')
  .then(() => console. log('MongoDB Connected'))
  .catch(err => console.error('MongoDB Error:', err));
// Import Routes
const authRoutes = require('./routes/auth'); const
```

```
// Database Connection
mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/attendance_db')
  .then(() => console. log('MongoDB Connected'))
  .catch(err => console.error('MongoDB Error:', err));
// Import Routes
const authRoutes = require('./routes/auth'); const
courseRoutes = require('./routes/course');
const attendanceRoutes = require('./routes/attendance');
// Mount Routes app.use('/api',
authRoutes);
app.use('/api/courses', courseRoutes);
app.use('/api/attendance', attendanceRoutes);
// Root Route
app.get('/', (req, res) => { res.json({
    message: 'Attendance Management System API', status:
    'running',
    routes: [
      'POST /api/register', 'POST
      /api/login', 'POST
      /api/courses', 'GET
      /api/courses',
      'POST /api/courses/:id/enroll',
      'POST /api/attendance/mark',
      'GET /api/attendance/my-attendance', 'GET
      /api/attendance/course/:id',
      'GET /api/attendance/course/:id/report'
    ]
  });
});
// 404 Handler app.use((req,
res) => {
  console.log('404 - Route not found:', req.method, req.url);
  res.status(404).json({
```

```
    error: 'Route not found',
    requestedUrl: req.url,
    method: req.method
  });
});
// Error Handler
app.use((err, req, res, next) => { console.error('Server
  Error:', err); res.status(500).json({ error: err.message
  });
});
// Start Server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`🚀 Server running on http://localhost:${PORT}`); console.log(`📚
  API Docs: http://localhost:${PORT}/`);
});
```

auth.js:

```
const jwt = require('jsonwebtoken');

// Verify JWT Token
const auth = async (req, res, next) => { try {
    const token = req.header('Authorization')?.replace('Bearer ', ''); if (!token)
    {
      return res.status(401).json({ error: 'No token provided' });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET); req.user =
    decoded;
    next();
  } catch (error) {
    res.status(401).json({ error: 'Invalid token' });
  }
};

// Check if user is a teacher
const isTeacher = (req, res, next) => { if
  (req.user.role !== 'teacher') {
    return res.status(403).json({ error: 'Access denied. Teachers only.' });
```

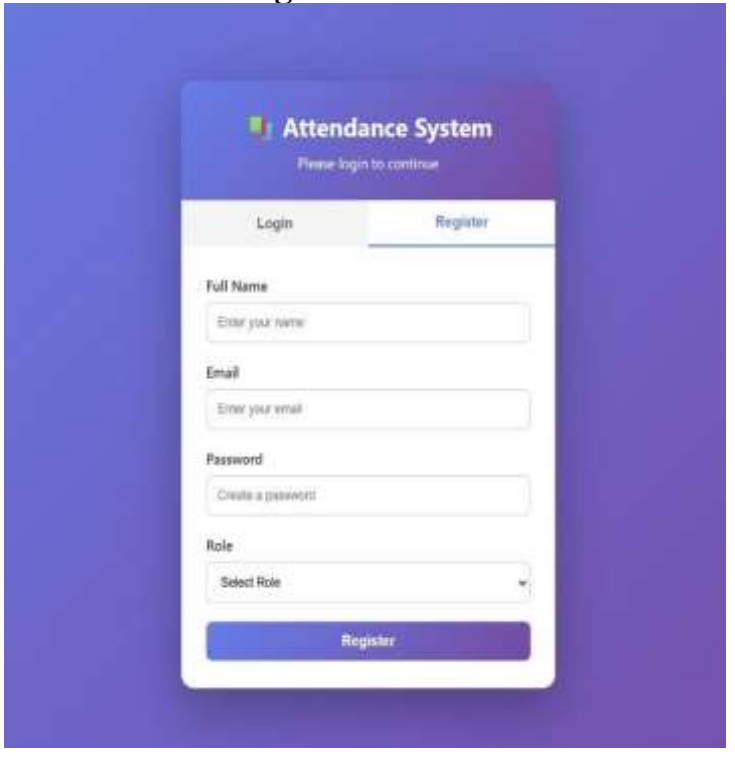
```
};

module.exports = { auth, isTeacher };
```

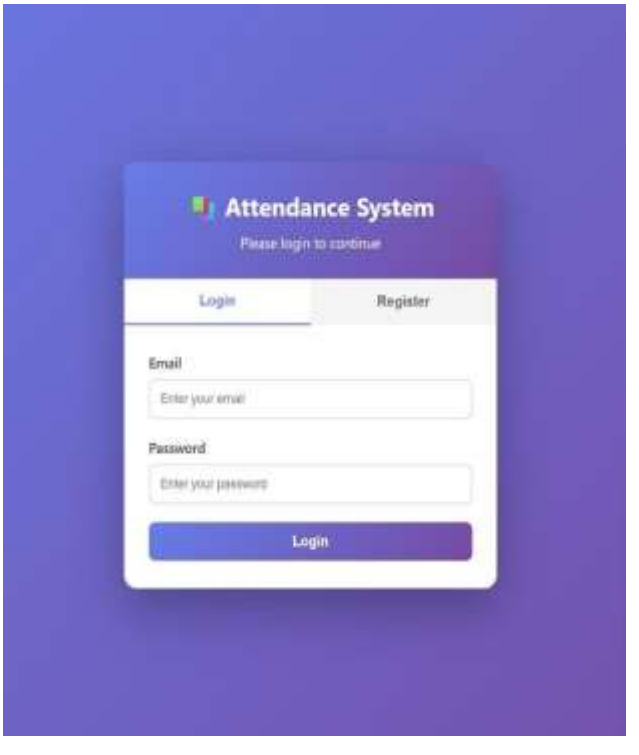
.env: MONGO_URI=mongodb://127.0.0.1:27017/attendance_db
JWT_SECRET=mysecretkey123
PORT=5000

Output:

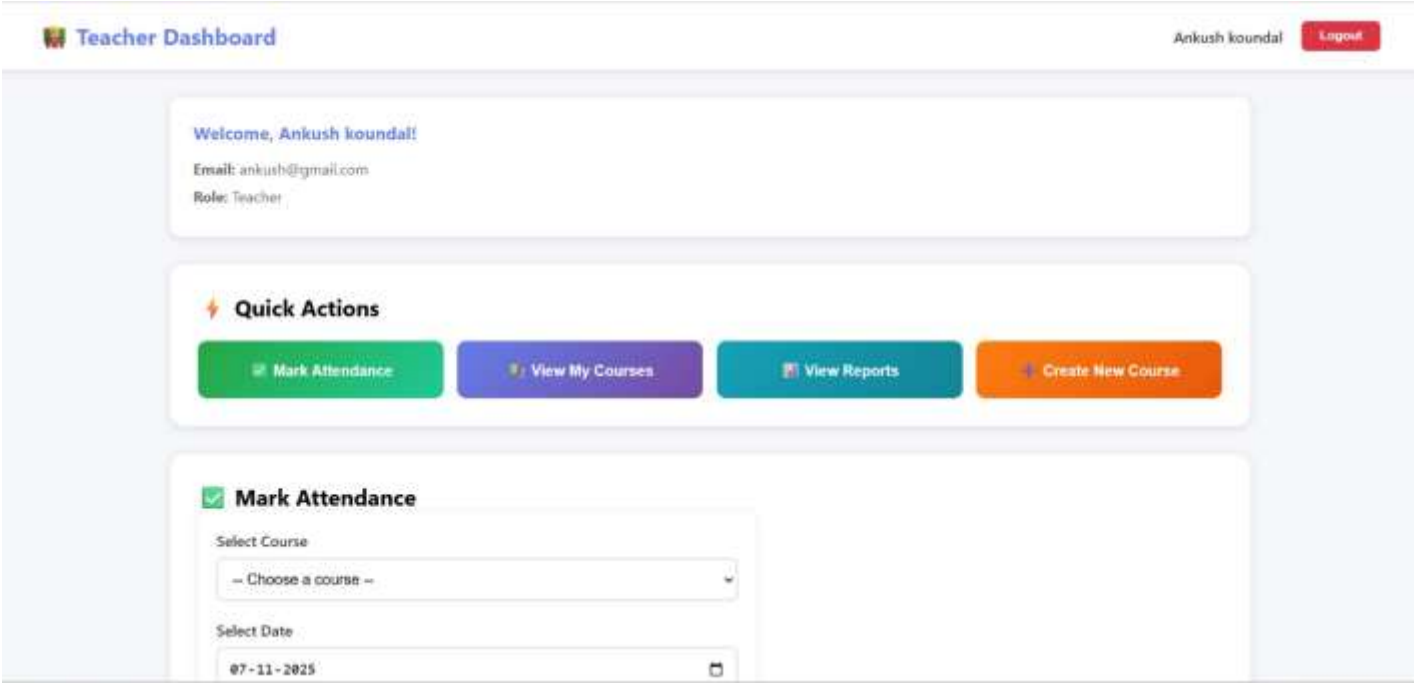
Registration Form



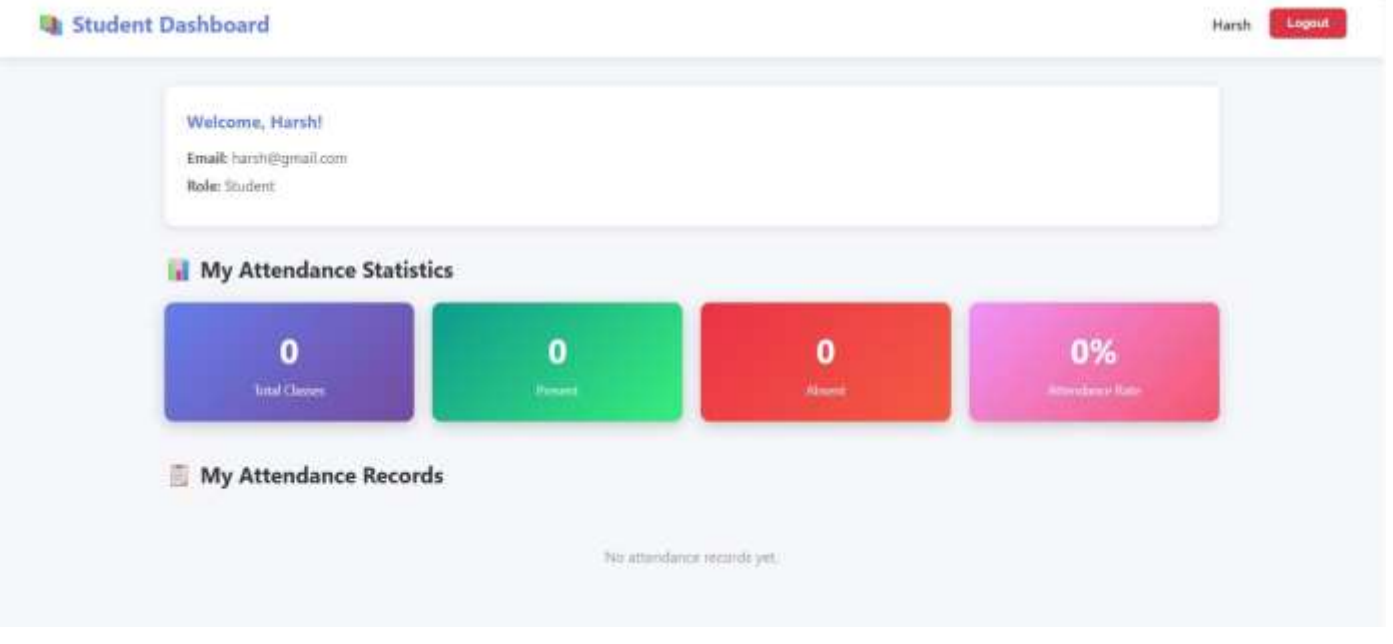
Login Form



Teacher Dashboard



Student Dashboard



4. Learning Outcomes:

1. Understood role-based access control – how to differentiate features for teachers and students securely.
2. Designed relational database structures for user, course, and attendance data (and maintain foreign key relationships).
3. Implemented data validation and integrity to prevent duplication or unauthorized access.
4. Enhanced problem-solving skills by converting real-world attendance needs into structured programming logic.
5. Integrate frontend and backend components for interactive data handling (e.g., attendance forms, dynamic data display).